

ADR-03: Estilo arquitectónico del proyecto

ZenFlow — Arquitectura en capas como columna vertebral del sistema

Campo	Valor
Autor	Dylan Vazquez
Fecha	15/05/2025
Estado	Aceptado

Contexto

Los ADR-01 y ADR-02 ya documentan decisiones puntuales — qué tecnología usar (WPF) y cómo persistir datos (JSON con interfaces). Pero ninguno de los dos formaliza la decisión que en realidad sostiene a las dos: el estilo arquitectónico general de ZenFlow. Este ADR existe para dejar esa decisión explícita, justificada y separada de las demás, porque es la que más impacto tiene sobre cómo se organiza todo el código del proyecto.

Las condiciones que influyen en esta decisión:

- ZenFlow tiene tres módulos funcionales claramente distinguibles: gestión de tareas, modo enfoque y seguimiento de hábitos. Cada uno necesita su propia lógica, pero todos comparten la misma necesidad de mostrar UI, aplicar reglas y guardar datos.
- El proyecto lo desarrollo solo, durante un semestre. La arquitectura elegida tiene que ser algo que pueda mantener mentalmente sin documentación externa extensa — si me pierdo en mi propio código, el proyecto se detiene.
- Ya aplico principios SOLID (SRP, OCP, DIP) en los ADR anteriores. El estilo arquitectónico tiene que ser compatible con esos principios, no competir con ellos.
- El primer prototipo (el ejercicio del Ahorcado) mostró en carne propia qué pasa sin una arquitectura definida: una sola clase con todo mezclado que se vuelve difícil de modificar conforme crece.

Decisión

Adoptar arquitectura en capas (Layered Architecture) como estilo arquitectónico de ZenFlow, con cuatro capas estrictamente jerárquicas: Presentación, Lógica de negocio, Repositorios y Datos.

La tabla siguiente resume cómo se organiza cada capa y cuál es la regla de dependencia que la gobierna:

Capa	Contenido en ZenFlow	Regla de dependencia
------	----------------------	----------------------

Presentación	WPF/XAML — VistaTablero, VistaEnfoque, VistaHábitos	Solo puede llamar a Lógica
Lógica	GestorTareas, MotorEnfoque, GestorHábitos	Solo puede llamar a Repositorios
Repositorios	ITareaRepo, IHábitoRepo (interfaces)	No llama a nadie — es el contrato
Datos	TareaRepoJson, HábitoRepoJson	Implementa Repositorios, toca el disco

¿Por qué?

La arquitectura en capas no se eligió porque sea la más sofisticada — se eligió porque resuelve exactamente el problema que tengo: necesito que cada módulo (tareas, hábitos, enfoque) tenga sus propias clases en cada capa, sin que se mezclen entre sí ni con la forma en que se guardan los datos.

La característica concreta que más me convence es la regla de dependencia unidireccional: cada capa solo puede llamar a la capa inmediatamente inferior, nunca saltarse niveles ni llamar hacia arriba. Esto significa que si abro cualquier archivo del proyecto, sé inmediatamente en qué capa estoy y qué cosas tiene permitido conocer esa clase. Esa previsibilidad es lo que necesito trabajando solo — no tengo que recordar reglas complejas, solo seguir la misma estructura para cada módulo nuevo.

Además, esta arquitectura es la que hace posibles las decisiones de los ADR-01 y ADR-02. Sin capas separadas, no tendría sentido hablar de 'interfaces de repositorio' — esa capa solo existe porque la arquitectura la define como un nivel propio.

Alternativas consideradas

Alternativa	Por qué la descarté
Arquitectura Hexagonal (Ports & Adapters)	Es muy similar en espíritu al DIP que ya aplico con las interfaces de repositorio, pero formalizarla completa implica agregar 'puertos' y 'adaptadores' como conceptos explícitos en todo el sistema, no solo en la persistencia. Para un proyecto de un semestre, esa formalidad agrega ceremonia sin un beneficio claro sobre lo que ya logro con capas + interfaces.
Microservicios	ZenFlow es una sola app de escritorio que corre en la máquina de un usuario. Microservicios resuelve problemas de sistemas distribuidos — escalar partes independientes, desplegar por separado, equipos distintos por servicio. Nada de eso aplica aquí. Habría sido sobreingeniería pura.
MVC (Model-View-Controller)	MVC es el patrón más común en apps web, pero WPF está diseñado alrededor de MVVM, no MVC. Forzar MVC en WPF

Arquitectura monolítica sin capas (todo en un archivo)	significa ir contra las herramientas del framework (data binding, dependency properties) en lugar de aprovecharlas. Es literalmente cómo empezó el primer prototipo del Ahorcado — una clase 'Juego' con todo mezclado. Funciona para algo pequeño, pero agregar el módulo de hábitos o el modo enfoque habría significado seguir creciendo esa clase única hasta volverla imposible de mantener.
--	---

Consecuencias

Lo que gano:

- Técnica: agregar un módulo nuevo (por ejemplo, una futura 'Agenda') sigue siempre el mismo patrón — una vista en Presentación, un Gestor en Lógica, una interfaz en Repositorios y una implementación en Datos. No tengo que inventar una estructura distinta cada vez, lo que reduce el riesgo de inconsistencias en el código.
- Proceso: como cada capa tiene una responsabilidad clara, cuando algo no funciona sé en cuál de las 4 capas buscar antes de abrir un solo archivo. Eso reduce el tiempo de debugging, que es crítico trabajando solo y con tiempo limitado.

Lo que sacrifico o asumo:

- Limitación técnica: la arquitectura en capas asume que el flujo de datos es relativamente lineal (UI → Lógica → Datos). Si en el futuro ZenFlow necesitara funcionalidades más complejas, como sincronización en tiempo real entre módulos o eventos que viajen 'hacia arriba' desde Datos hasta la UI, esta estructura estricta de capas se volvería incómoda y necesitaría patrones adicionales como eventos o mensajería.
 - Deuda y riesgo: existe la tentación de 'saltarse capas' cuando algo parece simple — por ejemplo, que VistaTablero llame directo a TareaRepoJson para ahorrar una línea de código. Si eso ocurre incluso una vez, rompe la regla de dependencia y empieza a degradar la arquitectura silenciosamente. Mantener la disciplina de respetar las capas es un esfuerzo constante, no algo que se garantiza solo por haberlo decidido aquí.
-

Diagrama

